

Exploration d’architectures neuronales profondes pour l’évaluation du risque ordinal

Noé Cochet

Dép. d’informatique et de génie logiciel
Université Laval, Québec, Canada
noe-pierre.cochet.1@ulaval.ca

Théo Fornier

Dép. d’informatique et de génie logiciel
Université Laval, Québec, Canada
theo.fornier.1@ulaval.ca

Bilal Kefif

Dép. d’informatique et de génie logiciel
Université Laval, Québec, Canada
bilal.kefif.1@ulaval.ca

Étienne Plante-Dubé

Dép. d’informatique et de génie logiciel
Université Laval, Québec, Canada
etienne.plante-dube.1@ulaval.ca

Jean-Simon Ratté

Dép. d’informatique et de génie logiciel
Université Laval, Québec, Canada
jean-simon.ratte.1@ulaval.ca

Résumé—Ce projet compare des méthodes de *gradient boosting* (XGBoost, CatBoost) à des architectures d’apprentissage profond récentes (TabNet, TabM) pour la classification ordinaire du risque en assurance-vie sur le jeu de données *Prudential Life Insurance Assessment*. Parmi les architectures profondes testées, TabM s’avère la plus compétitive. Pour cette approche, nous étudions les variantes *BatchEnsemble*, *packed* et *mini*, avec *embeddings PiecewiseLinear* et ajustement bayésien des hyperparamètres. Nous évaluons en complément l’apport de données synthétiques générées par CTGAN, TVAE, TabSyn et un *prompting* LLM (GPT-5.3), ainsi qu’une stratégie *teacher-student* avec pseudo-étiquetage filtré par confiance. CatBoost obtient le meilleur score (QWK de 0,662 en validation) suivi de près par TabM[†] (QWK de 0,661). Avec l’ajout de données synthétiques, TabM[†] parvient à obtenir un résultat QWK de 0,670, soit un gain modeste en validation. Notre étude d’ablation identifie les *embeddings PiecewiseLinear* et le *BatchEnsemble* comme principaux contributeurs des gains observés sur TabM[†]. Cela en fait un concurrent crédible du *gradient boosting*, qui demeure la référence sur ce type de problème tabulaire.

Index Terms—apprentissage profond, données tabulaires, classification ordinaire, assurance-vie, semi-supervisé, transformer, gradient boosting

I. INTRODUCTION

L’évaluation du risque lors de la souscription en assurance-vie repose traditionnellement sur de longs questionnaires médicaux destinés à déterminer l’éligibilité aux produits d’assurance. Depuis quelques années, les assureurs y ont intégré des algorithmes d’apprentissage automatique afin de réduire le recours aux examens médicaux complémentaires tout en limitant le risque additionnel de mortalité. Les données recueillies dans ces questionnaires comportent de nombreuses variables médicales, comportementales et démographiques. Le processus d’attribution d’une classe de risque à chaque client constitue ainsi un problème de classification ordinaire à huit niveaux.

Sur ce type de données tabulaires structurées, les méthodes de *gradient boosting* telles que XGBoost [1], LightGBM [2] et CatBoost [3] demeurent l’état de l’art. Pourtant, l’essor récent d’architectures de réseaux de neurones profonds conçues spé-

cifiquement pour les données tabulaires, comme les approches fondées sur l’attention et les modèles de fondation, laisse entrevoir un potentiel compétitif croissant pour l’apprentissage profond.

Ce projet, réalisé dans le cadre du cours GLO-7030, vise à évaluer dans quelle mesure les avancées récentes en apprentissage profond peuvent rivaliser avec les modèles de *boosting* classiques sur un problème réel d’évaluation du risque de mortalité en assurance-vie. Notre objectif principal est de comparer des architectures modernes d’apprentissage profond dédiées aux données tabulaires (TabNet, TabM), au travers d’une étude d’ablation sur les composantes d’entraînement, et d’explorer des stratégies semi-supervisées ainsi que la génération de données synthétiques.

La suite de ce rapport est organisée comme suit : la Section II présente l’état de l’art ; la Section III décrit l’approche proposée ; la Section IV détaille la méthodologie et les expérimentations ; la Section V présente et discute les résultats ; la Section VI ce qui n’a pas fonctionné, la Section VII des pistes d’amélioration ; et, finalement, la Section VIII conclut le rapport.

II. ÉTAT DE L’ART

A. Gradient Boosting pour les données tabulaires

Les méthodes de *gradient boosting* constituent depuis plusieurs années l’approche dominante pour les tâches d’apprentissage supervisé sur données tabulaires. XGBoost [1] introduit un cadre d’optimisation scalable avec régularisation L1/L2 intégrée. LightGBM [2] améliore l’efficacité computationnelle grâce à une construction d’arbres feuille par feuille et un échantillonnage par gradient. CatBoost [3] propose un traitement natif des variables catégorielles via un encodage ordonné.

B. Apprentissage profond sur des données tabulaires

Plusieurs travaux récents ont tenté de combler l’écart de performance entre les réseaux de neurones et les méthodes de

boosting sur les données tabulaires [4]. Nous distinguons trois familles d’approches pertinentes pour notre étude.

1) *Architectures attentionnelles spécialisées*: TabNet [5] est une architecture neuronale conçue spécifiquement pour les données tabulaires, fondée sur un encodeur séquentiel à étapes successives. À chaque étape de décision, un mécanisme d’attention *sparsemax* sélectionne un sous-ensemble parcimonieux de variables (*feature masking*), ce qui rend le modèle interprétable et limite le sur-ajustement en haute dimension. L’architecture combine une régularisation de parcimonie sur les masques à une perte supervisée standard, permettant un entraînement de bout en bout. Contrairement aux modèles de fondation, TabNet est entraîné directement sur le jeu de données cible.

2) *Approches d’ensemble paramétriquement efficaces*: TabM [6] combine un MLP avec un mécanisme de *BatchEnsemble* appliqué à chaque couche linéaire. Quelques adaptateurs légers transforment indépendamment le flux d’information, créant ainsi k représentations parallèles tout en partageant les poids principaux.

3) *Modèles de fondation tabulaires (non retenus)*: TabPFN [7] est un modèle de fondation *prior-fitted* entraîné sur des millions de jeux de données synthétiques bayésiens, permettant une inférence en contexte sans ré-entraînement. Dans notre cas, TabPFN n’a pas été retenu : la version originale est limitée à des jeux de données de quelques milliers d’échantillons, alors que *Prudential* en contient 59 381.

TabDPT [8] adapte l’architecture Transformer à l’apprentissage tabulaire via un pré-entraînement par *deep pseudo-target*, exploitant des données non étiquetées pour améliorer les représentations apprises. TabDPT n’a pas été évalué dans le cadre de ce projet et constitue une piste d’amélioration discutée en Section VII.

C. Classification ordinale

La prédiction d’une variable cible catégorique ordonnée, telle que les niveaux de risque, nécessite des ajustements par rapport à la classification multi-classe standard pour préserver la hiérarchie entre les étiquettes. Une approche séminale consiste à transformer le problème en une série de tâches binaires cumulatives, permettant de modéliser la probabilité que la cible dépasse un seuil donné [9].

Au-delà de la décomposition des classes, l’enjeu réside dans l’optimisation de fonctions de perte sensibles à l’ordre. La *Earth Mover’s Distance* (EMD), ou distance de Wasserstein, est particulièrement adaptée puisqu’elle pénalise les erreurs proportionnellement à la distance entre la classe prédite et la classe réelle [10]. Dans le contexte actuariel, cette approche est souvent complétée par l’optimisation directe ou indirecte du *Quadratic Weighted Kappa* (QWK), qui mesure l’accord entre les évaluateurs en tenant compte de l’ampleur des désaccords.

D. Apprentissage semi-supervisé et augmentation tabulaire

L’apprentissage semi-supervisé offre des avenues prometteuses pour pallier la rareté des données étiquetées dans les domaines complexes. Pour les données tabulaires, cela se traduit souvent par le pseudo-étiquetage, où un modèle

prédicatif assigne des étiquettes provisoires aux données non étiquetées afin d’augmenter l’ensemble d’entraînement [11]. Une autre méthode performante est le pré-entraînement auto-supervisé par auto-encodage, comme illustré par l’architecture VIME (*Variational Information Maximizing Self-Supervised Learning*) [12], qui apprend des représentations robustes en reconstruisant des données corrompues.

Enfin, le paradigme *teacher-student*, initialement conçu pour la distillation de connaissances, permet de transférer l’information apprise par un modèle complexe (le maître) vers un modèle plus léger ou entraîné sur des données synthétiques (l’élève) [13]. Cette stratégie est particulièrement pertinente lors de l’utilisation de techniques d’augmentation tabulaire, où le modèle doit apprendre à généraliser à partir d’échantillons générés artificiellement pour équilibrer les classes minoritaires.

III. APPROCHE PROPOSÉE

A. Vue d’ensemble

Notre approche suit un cadre expérimental structuré en quatre volets interdépendants : (1) l’établissement de lignes de base robustes à l’aide de modèles de *gradient boosting* optimisés ; (2) l’exploration d’architectures d’apprentissage profond de pointe, telles que TabNet et TabM, adaptées à la nature ordinale des risques ; (3) une stratégie de génération de données synthétiques pour quantifier le gain de performance apporté par l’ajout de données synthétiques dans un contexte de données restreintes et de classes déséquilibrées ; et (4) une étude d’ablation systématique pour l’architecture retenue afin d’évaluer l’apport spécifique de chaque composante architecturale de celle-ci.

B. Description du jeu de données

Le jeu de données *Prudential Life Insurance Assessment* [14] est composé de 59 381 exemples d’entraînement et 19 765 exemples de test, chacun décrit par 128 variables couvrant des caractéristiques médicales, comportementales et démographiques. La variable cible *Response* prend 8 valeurs ordinales (1 à 8). La distribution des classes est **fortement déséquilibrée, avec une prédominance de la classe 8**.

Les variables peuvent être regroupées en :

- Variables continues (ex. : Ht, Wt, BMI) ;
- Variables catégorielles à haute cardinalité (ex. : *Product_Info_2*) ;
- Variables binaires de conditions médicales (ex. : *Medical_History_**) ;
- Variables avec valeurs manquantes (ex. : *Employment_Info_**).

C. Prétraitement

Pour les modèles de *gradient boosting*, l’imputation des valeurs manquantes des variables numériques continues a été réalisée par la médiane. Les variables catégorielles à haute cardinalité ont été transformées par *target encoding* (encodage par la moyenne de la cible) afin de modéliser les relations complexes tout en limitant la dimensionnalité de l’espace des caractéristiques. Les variables présentant un taux de valeurs manquantes supérieur à un seuil critique (par ex.

Décomposition binaire pour le *boosting* : l'approche classique de régression a été substituée par une architecture

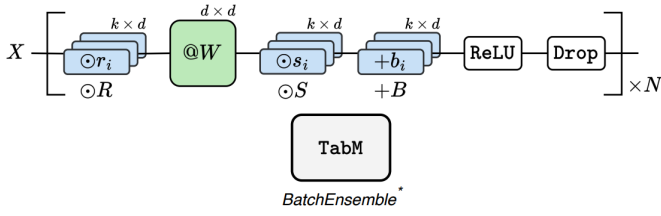


FIGURE 4. TabM. Tiré de Gorishniy et al. [6].

de classification ordinaire inspirée des meilleures solutions de la compétition, adaptée ici à XGBoost et à CatBoost. La classification ordinaire est décomposée en sept classifieurs binaires prédisant successivement la probabilité de dépassement d'un seuil de classe ($y > 1, y > 2, \dots, y > 7$). La somme de ces probabilités (augmentée de 1) fournit une prédiction continue. Plutôt que d'employer des seuils de discrétisation globaux, une calibration conditionnelle (*variable split calibration*) est appliquée en partitionnant l'ensemble des prédictions selon la variable binaire `Medical_History_23`. Pour chaque sous-ensemble, les seuils sont initialisés à partir des percentiles de la distribution, puis optimisés par recherche locale (*grid search*) afin de maximiser le QWK de validation. Couplée aux probabilités issues de CatBoost, cette optimisation des seuils apporte un gain absolu de QWK d'environ 0,06 par rapport à un arrondissement scalaire standard.

Pour adapter les modèles d'apprentissage profond à la nature ordinaire de la cible (8 classes ordonnées, $y \in \{1, \dots, 8\}$), nous adoptons une approche de **régression ordinaire continue** plutôt qu'une classification multi-classes standard.

La tête de sortie des deux modèles produit un scalaire réel ($d_{\text{out}} = 1$). La variable cible est standardisée avant l'entraînement :

$$\tilde{y} = \frac{y - \mu_y}{\sigma_y}, \quad (1)$$

où μ_y et σ_y sont estimés sur l'ensemble d'entraînement. À l'inférence, la prédiction est dénormalisée puis arrondie à la classe ordinaire la plus proche.

TabNet utilise la MSE comme fonction de perte. TabM, en revanche, optimise directement la métrique d'évaluation via une approximation différentiable du *Quadratic Weighted Kappa* (QWK), la soft-QWK.

Le QWK est défini à partir d'une matrice de pondération quadratique qui pénalise les erreurs proportionnellement à leur écart ordinal :

$$W_{ij} = \frac{(i - j)^2}{(K - 1)^2}, \quad i, j \in \{1, \dots, K\}. \quad (2)$$

La soft-QWK s'appuie sur une softmax pour relâcher la distribution et fait intervenir un hyperparamètre de température τ qui contrôle la netteté de la distribution sur les classes : un τ faible place presque tout le poids sur la classe la plus proche, tandis qu'un τ élevé l'étale sur les classes voisines.

Dans les deux cas, l'*early stopping* est piloté par le QWK mesuré sur le jeu de validation à chaque époque.

G. Utilisation de données synthétiques

Dans notre étude, nous avons été confrontés à plusieurs limitations classiques : taille limitée du jeu de données, déséquilibre entre les classes et difficulté à modéliser des relations complexes entre les variables.

Les modèles d'apprentissage profond sont sensibles à la quantité de données disponibles [16] : leur performance tend à augmenter avec le volume de données d'entraînement, contrairement à certains modèles tabulaires classiques (comme les méthodes de *boosting*), souvent plus robustes en régime de faibles données.

Afin de pallier ces limitations, nous avons choisi d'explorer l'utilisation de données synthétiques pour enrichir le jeu de données initial. Notre approche repose sur un pipeline structuré en plusieurs étapes :

- 1) analyse du jeu de données initial (types de variables, distributions, corrélations) ;
- 2) génération de données synthétiques conditionnées sur certaines classes ;
- 3) évaluation de la cohérence des données générées ;
- 4) augmentation du jeu de données réel ;
- 5) entraînement et comparaison des modèles.

Plusieurs modèles génératifs adaptés aux données tabulaires ont été étudiés.

1) *CTGAN*: Le modèle CTGAN [17] est une extension des GAN spécifiquement conçue pour les données tabulaires. Il permet de gérer les variables mixtes (numériques et catégorielles) via un mécanisme de conditionnement. Nous l'utilisons pour générer des données synthétiques conditionnées par classe, afin de mieux modéliser les distributions spécifiques à chacune.

2) *TVAE*: Le modèle TVAE (*Tabular Variational Autoencoder*) [17] repose sur une approche probabiliste permettant de modéliser la distribution conjointe des variables. La même stratégie de génération conditionnée par classe est appliquée avec TVAE.

3) *TabSyn*: Nous avons également expérimenté TabSyn [18], une approche fondée sur des modèles de diffusion en espace latent pour la génération de données tabulaires. L'implémentation utilisée provient du dépôt officiel que nous avons cloné et adapté à notre jeu de données. Cette approche permet elle aussi une génération conditionnée afin de capturer des distributions propres à chaque classe.

4) *Génération via LLM*: Nous avons également exploré l'utilisation de modèles de langage (LLM) comme générateurs conditionnels, à partir de *prompts* structurés décrivant le format et les contraintes du jeu de données. En plus du *prompt*, nous avons fourni au LLM le jeu de données d'entraînement afin qu'il puisse mieux saisir la structure des données, les types de variables, les plages de valeurs et les relations entre les caractéristiques. Cette approche permet de produire des données synthétiques en contrôlant explicitement certaines caractéristiques via le *prompt*, notamment la classe cible et la structure des variables. Un exemple de *prompt* utilisé pour générer des données synthétiques est présenté en annexe X-A.

Plusieurs modèles ont été testés, notamment GPT-5.3, Gemini 3.1 Pro et Claude Opus 4.7. Parmi ceux-ci, GPT-5.3 a produit les résultats les plus satisfaisants dans notre cadre expérimental. Les modèles testés correspondent à des modèles présentés officiellement par OpenAI, Google DeepMind et Anthropic [19]–[21].

Pour chacune de ces méthodes, un modèle distinct a été entraîné pour chaque classe afin de générer séparément des données appartenant à cette classe.

On peut retrouver l’architecture globale utilisée pour la génération de données synthétiques à la Figure 5. Cette architecture reste la même pour les différentes expériences ; seul le générateur change, celui-ci pouvant être l’un des modèles testés parmi ceux cités ci-dessus.

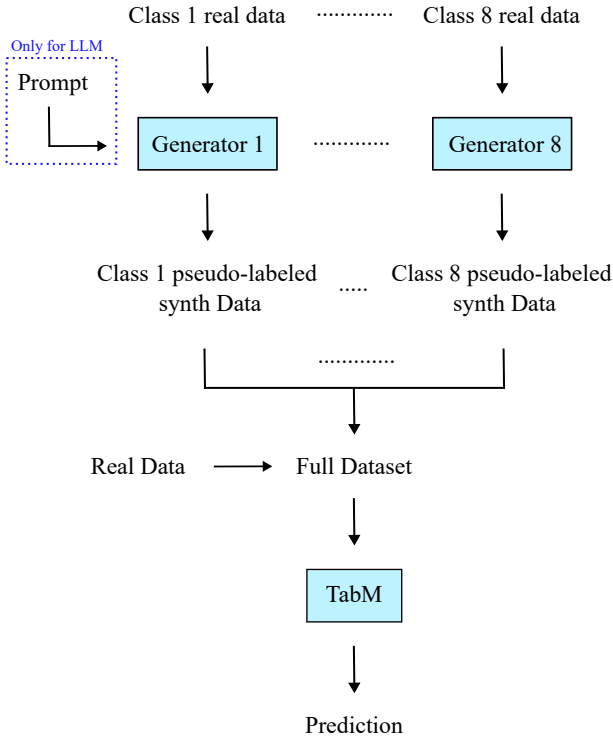


FIGURE 5. Approche de génération de données synthétiques

5) *Pseudo-labellisation teacher-student*: Nous avons également testé une seconde approche pour exploiter les données synthétiques. Les modèles de génération ne semblaient pas améliorer les résultats lorsqu’ils généraient directement les données ainsi que leurs étiquettes, c’est-à-dire en se basant sur une distribution du type $p(\text{donnée} \mid \text{étiquette})$. Nous avons donc décidé de générer les données sans leurs étiquettes, en nous basant uniquement sur une distribution du type $p(\text{donnée})$.

Au lieu d’utiliser un modèle de génération entraîné séparément pour chaque classe, nous avons donc entraîné le modèle de génération sur l’ensemble du jeu de données. Les données synthétiques produites ne possédant pas d’étiquette, nous avons ensuite utilisé une approche de pseudo-labellisation de type *teacher-student*.

Le principe général de cette approche est d’utiliser un ou plusieurs modèles *teacher* pour attribuer des pseudo-labels à des données non annotées. Ces pseudo-labels sont ensuite utilisés comme annotations supplémentaires pour entraîner un modèle *student*. Dans notre cas, les modèles *teacher* sont XGBoost et CatBoost. Le modèle *student* est le modèle de deep learning TabM, car il s’agit du meilleur modèle de deep learning évalué, comme montré dans la section V, et que cette étude vise à analyser l’utilité des modèles de deep learning. Une première version de cette approche consistait à pseudo-labelliser directement l’ensemble des données synthétiques à l’aide des modèles de boosting. Cette stratégie n’a pas permis d’améliorer les performances. Nous avons observé un phénomène d’auto-renforcement : les pseudo-labels générés pouvaient introduire du bruit dans le jeu d’entraînement, ce qui dégradait ensuite les performances du modèle *student*.

Nous avons donc retenu une approche plus conservatrice, consistant à ne conserver que les données synthétiques pour lesquelles les deux modèles *teacher* étaient suffisamment cohérents et confiants. L’objectif n’était pas d’ajouter le plus grand nombre possible de données synthétiques, mais d’ajouter uniquement des pseudo-labels suffisamment fiables pour ne pas dégrader l’entraînement final.

Dans cette partie, nous avons choisi d’utiliser GPT-5.3 comme générateur de données synthétiques. Ce choix est justifié par les résultats expérimentaux présentés dans la section V, où cette méthode obtient les meilleures performances parmi les approches testées.

On peut retrouver l’architecture globale utilisée pour l’approche *teacher-student* à la Figure 6.

IV. MÉTHODOLOGIE ET EXPÉRIMENTATION

A. Protocole d’évaluation

Tous les modèles sont évalués selon les métriques suivantes :

- **Kappa de Cohen quadratique pondéré (QWK)** : métrique principale du concours Kaggle ;
- **Accuracy** : taux de classification exacte ;

Nous utilisons une validation croisée stratifiée à $k = 5$ plis pour tous les modèles supervisés qui n’utilisent pas de données synthétiques. Les résultats sur l’ensemble de test Kaggle sont obtenus via soumission.

B. Détails d’entraînement

L’optimisation des hyperparamètres pour XGBoost et CatBoost a été réalisée par une recherche bayésienne (*bayesian sweep*) de 100 itérations orchestrée par *Weights & Biases* (W&B). Les modèles ont été évalués sur une séparation stricte (*split* 80/20) des données nettoyées, en utilisant le QWK de l’ensemble de validation comme critère d’arrêt précoce (*early stopping*) après 50 itérations sans amélioration. L’entraînement a été effectué sur le *cluster* de calcul de l’Université Laval (nœuds ICE) avec accélération matérielle.

Pour les modèles de deep learning, les entraînements ont été faits sur un ordinateur avec un GPU NVIDIA RTX 2000 Ada Generation avec 8GB de mémoire ainsi qu’un CPU Intel i7-13700HX de 13^e génération.

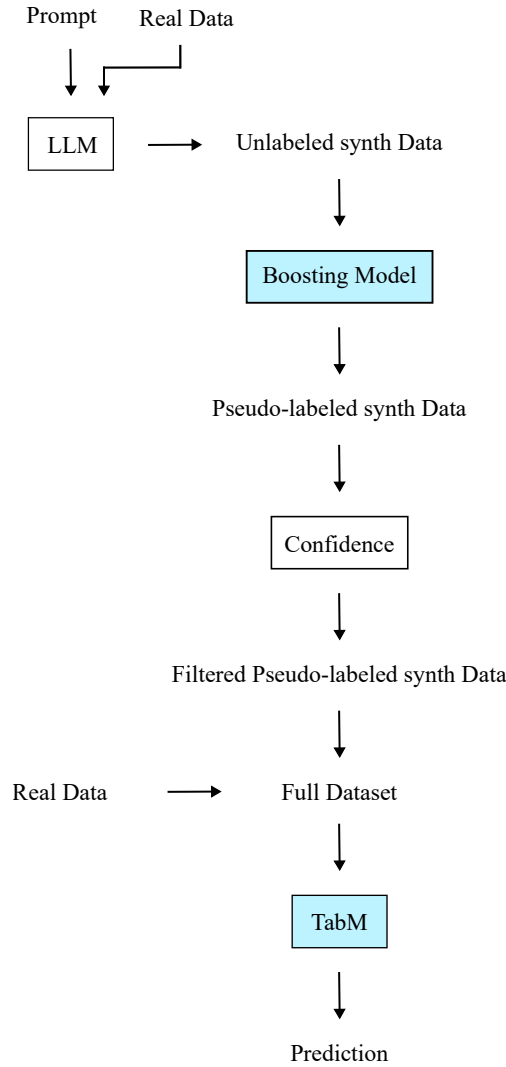


FIGURE 6. Approche finale de pseudo-labellisation teacher-student

C. Méthodologie d'entraînement des modèles de boosting

L'entraînement des modèles de *gradient boosting* (XGBoost et CatBoost) suit un pipeline spécifiquement optimisé pour la régression ordinale. L'apprentissage ne se fait pas directement sur le QWK, mais repose sur la décomposition du problème en sept tâches de classification binaire indépendantes (prédisant si $y > k$ pour $k \in \{1, \dots, 7\}$). Chaque sous-modèle est optimisé selon l'entropie croisée binaire (*logloss*).

1) *Prétraitement dynamique*: Une particularité de l'approche réside dans l'extraction dynamique de caractéristiques. Pour chaque tâche binaire, un encodage cible (*target encoding*) est calculé spécifiquement pour la sous-tâche en cours sur les variables `Medical_Keyword_*`. La moyenne de la cible binaire est substituée aux valeurs positives, permettant d'enrichir la représentation des antécédents médicaux sans introduire de fuite de données d'une classe à l'autre.

2) *Recherche d'hyperparamètres et protocole d'évaluation*: L'optimisation des hyperparamètres a été menée via une

recherche bayésienne de 100 itérations, avec une allocation favorisant CatBoost (70 % des essais) car ce modèle était le plus performant, donc on se concentre sur celui-ci afin de maximiser le score. L'évaluation des combinaisons s'est faite sur une séparation stratifiée 80/20 du jeu de données. Lors de cette étape, un *early stopping* de 50 itérations est utilisé sur l'ensemble de validation. Contrairement aux approches d'apprentissage profond évaluées par la suite, le *boosting* ne recourt pas à une validation croisée à 5 plis, la séparation 80/20 s'avérant suffisante pour garantir une généralisation robuste.

3) *Calibration (Variable Split Calibration)*: Pour obtenir la prédiction finale, les sept probabilités issues des modèles binaires sont sommées et décalées de +1, produisant un score continu dans l'intervalle $[1, 8]$. L'assignation de la classe finale repose ensuite sur une calibration conditionnelle scindée selon la valeur de la variable `Medical_History_23`. Pour chaque sous-groupe, les seuils de décision sont initialisés sur la base des percentiles de distribution attendus, puis finement ajustés via une recherche locale itérative dans le but de maximiser directement le *Quadratic Weighted Kappa* (QWK).

4) *Entraînement final*: Afin de générer les prédictions sur l'ensemble de test, la stratégie retenue consiste à réentraîner l'ensemble des sept classifieurs sur l'intégralité des données d'entraînement (100 %) en utilisant les hyperparamètres optimaux. Les seuils de calibration sont ensuite recalculés sur ces prédictions complètes avant d'être appliqués aux scores continus de l'ensemble de test.

D. Méthodologie d'entraînement des modèles de deep learning

Les paramètres et hyperparamètres mentionnés dans cette section pour l'entraînement des modèles de deep learning sont définis dans le fichier `config.py` du projet.

1) *Séparation des données*: Les données d'entraînement sont divisées de manière stratifiée selon la variable cible afin de préserver la distribution des huit classes dans chaque partition. En mode standard, une division 80/20 est effectuée : 80 % des données servent à l'entraînement et 20 % à la validation. La random seed est fixée à 42 pour assurer la reproductibilité des expériences.

En mode validation croisée, un *StratifiedKFold* à 5 plis est employé. Chaque pli est entraîné indépendamment avec une instance de modèle distincte. Les prédictions hors-pli (*out-of-fold*) sont agrégées sur l'ensemble des cinq plis et les prédictions sur le jeu de test sont moyennées sur les cinq modèles entraînés.

Il est important de préciser que la validation croisée est seulement utilisée lorsqu'il n'y a pas de données synthétiques car ces dernières ne représentent pas de vraies observations indépendantes, donc la validation croisée n'y apporte pas les garanties statistiques attendues.

2) *Gestion des données synthétiques dans l'entraînement*: Les données synthétiques sont intégrées uniquement dans l'ensemble d'entraînement. Plusieurs paramètres contrôlent leur utilisation :

— `max_synth_ratio = 1,0` : plafond du nombre d'exemples synthétiques par classe, exprimé comme

multiple du nombre d'exemples réels de cette classe dans le pli d'entraînement. Une valeur de 1,0 signifie qu'on ne peut pas ajouter plus d'exemples synthétiques d'une classe qu'il n'y a d'exemples réels de cette classe. Ce plafonnement par classe évite d'amplifier davantage le déséquilibre entre classes.

- `synth_sample_weight = 0,2` : poids attribué à chaque exemple synthétique dans la fonction de perte, par rapport aux données réelles. Une valeur de 0,2 signifie que chaque donnée synthétique contribue cinq fois moins qu'une donnée réelle au gradient.
- `max_synth_val_ratio = 0,0` : proportion maximale d'exemples synthétiques pouvant être ajoutés à l'ensemble de validation. La valeur 0,0 désactive cette option ; la validation est effectuée exclusivement sur des données réelles afin de garantir une évaluation non biaisée des performances.

Le paramètre `max_synth_val_ratio` n'a finalement pas été utilisé pour les tests finaux. En effet, l'ajout de données synthétiques à l'ensemble de validation introduit une forme de fuite de données (*data leakage*) : le modèle générateur ayant été entraîné sur les données d'entraînement, les exemples synthétiques ne constituent pas des observations véritablement indépendantes. La validation est donc effectuée exclusivement sur des données réelles.

3) *Pondération des classes*: Un mécanisme de pondération par fréquence inverse (`use_class_weights`) a été implémenté et testé pour atténuer le déséquilibre entre classes. Le poids d'un exemple de classe c est calculé selon :

$$w_c = \left(\frac{N}{K \cdot n_c} \right)^p \quad (3)$$

où N est le nombre total d'exemples, $K = 8$ le nombre de classes, n_c l'effectif de la classe c , et p l'exposant (`class_weight_power = 0,7`) contrôlant l'intensité de la correction. Un p de 0 revient à ne pas pondérer, tandis qu'un p de 1 applique la correction complète. Les poids sont normalisés pour conserver l'échelle de la perte.

Cependant, cette pondération s'est révélée redondante avec la fonction de perte *soft QWK* (décrite à la section III-F), qui intègre déjà implicitement le coût ordinal entre classes. **Elle est donc désactivée par défaut et n'a pas été utilisée pour les tests finaux.**

4) *Prétraitement des features*: Les variables continues sont normalisées à l'aide d'un `StandardScaler` ajusté exclusivement sur les données d'entraînement réelles (sans les données synthétiques). La même transformation est appliquée aux ensembles de validation et de test. La variable cible est également centrée-réduite lors de l'entraînement et les prédictions sont dénormalisées avant le calcul des métriques d'évaluation. Il est important de ne pas inclure les données synthétiques dans l'ajustement du `StandardScaler` pour ne pas biaiser la normalisation avec des données artificielles.

5) *Entraînement avec TabNet*: L'optimiseur utilisé est *Adam* et le taux d'apprentissage suit un calendrier *StepLR*.

Voici les paramètres et hyperparamètres utilisés pour la majorité des entraînements avec TabNet.

Hyperparamètre	config.py	Valeur
Features dim.	<code>n_d</code>	64
Attention dim.	<code>n_a</code>	64
Decision steps	<code>n_steps</code>	6
Gamma (γ)	<code>gamma</code>	1,5
Blocs GLU indépendants	<code>n_independent</code>	2
Blocs GLU partagés	<code>n_shared</code>	2
Momentum BatchNorm	<code>momentum</code>	0,02
Taux d'apprentissage	<code>lr</code>	10^{-3}
Batch size	<code>batch_size</code>	1024
Virtual batch size	<code>virtual_batch_size</code>	256
Époques max.	<code>epochs</code>	200
Patience (early stopping)	<code>early_stopping_patience</code>	7

TABLE I

HYPERPARAMÈTRES DU MODÈLE TABNET.

La fonction de perte utilisée est *MSE*, tel que mentionné dans la section III-F.

6) *Entraînement avec TabM*: L'optimiseur utilisé est *AdamW* et le taux d'apprentissage suit un calendrier *CosineAnnealingLR*.

Voici les paramètres et hyperparamètres utilisés pour la majorité des entraînements avec TabM.

Hyperparamètre	config.py	Valeur
Multi-sample predictions	<code>k</code>	32
Blocs résiduels	<code>n_blocks</code>	3
Dimension cachée	<code>d_block</code>	512
Dropout	<code>dropout</code>	0,25
Taux d'apprentissage	<code>lr</code>	3×10^{-4}
Weight decay	<code>weight_decay</code>	10^{-3}
Batch size	<code>batch_size</code>	512
Époques max.	<code>epochs</code>	100
Patience (early stopping)	<code>early_stopping_patience</code>	7
Fonction de perte	<code>loss_fn</code>	<code>soft_qwk</code>
Temp. soft QWK	<code>qwk_temperature</code>	0,5
Embeddings PLR	<code>use_embeddings</code>	True

TABLE II

HYPERPARAMÈTRES DU MODÈLE TABM.

Initialement, la fonction de perte *MSE* était utilisée, puis elle a été remplacée par la *Soft-QWK*. Il est possible de la modifier directement dans la configuration en attribuant la valeur *mse* au paramètre `loss_fn` du modèle TabM.

E. Méthodologie de génération des données synthétiques

Pour chaque méthode de génération (CTGAN, TVAE, TabSyn, LLM), nous adoptons une spécialisation par classe : le jeu d'entraînement est séparé selon la variable cible et un modèle distinct est entraîné pour chacune des huit classes. Cette approche permet de générer des données conditionnées de façon explicite, en sélectionnant directement la classe cible via le modèle utilisé lors de la génération.

1) *Stratégies de génération*: Plusieurs stratégies ont été mises en place afin d'évaluer l'impact de l'ajout de données synthétiques.

a) *Stratégie A : génération ciblée*: La stratégie A consiste à ne générer des données que pour les classes sous-représentées. Le nombre d'exemples à produire est déterminé à partir de la distribution initiale : les classes minoritaires sont augmentées

jusqu'à atteindre 50 % de la taille de la classe majoritaire. Cette stratégie vise à réduire le déséquilibre sans modifier fortement la distribution globale.

Pour chaque classe sélectionnée, le modèle correspondant est chargé, puis un nombre spécifique d'échantillons est généré. Les données synthétiques produites sont ensuite :

- associées explicitement à leur classe cible ;
- dotées de nouveaux identifiants afin d'éviter tout conflit avec les données réelles ;
- réorganisées pour correspondre exactement à la structure du jeu de données original.

Les données générées pour chaque classe sont sauvegardées séparément, concaténées pour former un jeu de données synthétique global, puis fusionnées avec les données réelles afin de produire un jeu de données augmenté.

b) Stratégie B : équilibrage complet: La stratégie B vise à générer des données synthétiques pour toutes les classes afin d'obtenir un jeu de données équilibré. Le nombre d'exemples générés est calculé de manière à atteindre un effectif similaire entre classes. Le processus de génération est identique à celui de la stratégie A :

- chargement du modèle associé à chaque classe ;
- génération du nombre requis d'échantillons ;
- attribution de nouveaux identifiants ;
- alignement de la structure avec le jeu de données réel.

Les données synthétiques sont ensuite concaténées et fusionnées avec le jeu initial pour produire une version entièrement équilibrée.

c) Stratégie C : classes 1 et 2: La stratégie C cible les classes pour lesquelles le modèle d'apprentissage profond présente le plus d'erreurs de prédiction, soit les classes 1 et 2. L'objectif est d'améliorer la performance du modèle sur ces classes en générant 20 000 exemples supplémentaires à l'aide des modèles correspondants. Comme pour les autres stratégies, les données synthétiques sont associées à leur classe cible, dotées de nouveaux identifiants et alignées avec la structure du jeu de données réel avant d'être fusionnées avec les données d'origine.

d) Stratégie D : classes 4, 6 et 7: La stratégie D repose sur le même principe en ciblant les classes 4, 6 et 7, identifiées lors des premiers tests comme celles dont les résultats semblaient le plus s'améliorer avec l'ajout de données synthétiques. Le processus de génération et d'intégration est identique à celui des autres stratégies : génération conditionnée par classe, attribution de nouveaux identifiants et fusion avec le jeu de données réel.

2) *Pseudo-labellisation teacher-student:* Le dataset non annoté a été généré par batchs de 25 000 données, pour obtenir un dataset global de 300 000 données non annotées. Nous avons utilisé GPT 5.3 pour générer ces données, étant donné que cette méthode avait obtenu les meilleurs résultats dans la partie précédente. À chaque génération, nous fournissons au LLM un prompt de génération, le dataset d'entraînement ainsi que les données déjà générées, afin d'éviter les doublons.

Pour chaque donnée synthétique non annotée, XGBoost et CatBoost prédisent une distribution de probabilités sur les classes possibles. À partir de ces distributions, nous récupérons

la classe prédite par chaque modèle, la probabilité maximale associée à cette classe, ainsi que la marge de confiance, définie comme l'écart entre les deux probabilités les plus élevées.

Une donnée synthétique est conservée uniquement si les prédictions des deux modèles sont identiques, ou suffisamment proches, avec un écart maximal d'une classe. Nous calculons ensuite la confiance moyenne des deux modèles à partir de leur probabilité maximale respective. Les données jugées trop incertaines sont rejetées.

Un premier seuil de confiance uniforme fixé à 0,75 a été testé. Ce seuil permettait de ne conserver que les pseudo-labels les plus fiables. Cependant, il conduisait à une forte concentration des données conservées dans la classe 8. Environ 80 % des données pseudo-labellisées provenaient alors de cette classe, tandis que les autres classes étaient beaucoup moins représentées. Comme la classe 8 est déjà la classe majoritaire dans le jeu de données réel, ce résultat n'était pas souhaitable, car il risquait de renforcer davantage le déséquilibre initial.

Nous avons donc modifié le critère de sélection des pseudo-labels. Au lieu d'utiliser un seuil uniforme de 0,75 pour toutes les classes, nous avons abaissé le seuil de confiance à 0,55 pour les classes sous-représentées, tout en conservant un seuil plus strict de 0,75 pour les classes 5 et 8. Cette modification permet de conserver davantage d'échantillons synthétiques associés aux classes minoritaires, sans relâcher excessivement le filtrage pour les classes déjà fortement représentées.

Après filtrage, nous avons également limité le nombre maximal d'échantillons synthétiques à 3000 par classe afin d'éviter que les classes dominantes prennent trop d'importance dans le jeu d'entraînement augmenté. La répartition finale des données synthétiques pseudo-labellisées conservées est la suivante :

- Classe 1 : 1659 échantillons
- Classe 2 : 3000 échantillons
- Classe 4 : 4 échantillons
- Classe 5 : 1364 échantillons
- Classe 6 : 3000 échantillons
- Classe 7 : 3000 échantillons
- Classe 8 : 3000 échantillons

Au total, le jeu de données synthétique pseudo-labellisé final contient 16023 échantillons.

L'absence de pseudo-labels pour la classe 3 et le très faible nombre d'échantillons pour la classe 4 constituent une limite de cette approche. Cette limite reste toutefois acceptable dans notre cas, car les données synthétiques pseudo-labellisées ne remplacent pas le jeu d'entraînement réel. Elles sont seulement ajoutées aux données réelles annotées. Les classes peu ou pas représentées dans les pseudo-labels restent donc apprises à partir du jeu de données réel.

Les données conservées sont finalement sauvegardées avec leur pseudo-label, leur score de confiance et leur marge, puis ajoutées aux données réelles annotées pour entraîner le modèle *student*.

TABLE III

COMPARAISON DES PERFORMANCES DES MODÈLES. LA COLONNE « BRUT » DONNE LE QWK OBTENU DIRECTEMENT À PARTIR DES PRÉDICTIONS ARRONDIES ; LA COLONNE « +OFFSET » APPLIQUE UNE OPTIMISATION D'OFFSETS PAR CLASSE QUI MAXIMISE LE QWK SANS RÉ-ENTRAÎNEMENT

Modèle	QWK _{brut}	QWK _{+offset}
XGBoost	0,566	0,621
CatBoost	0,608	0,662
MLP	0,594	0,632
TabNet	0,437	0,529
TabM	0,607	0,645
TabM(+ synth)	0,667	0,670
TabM(Teacher-student)	0,656	0,660
TabM [†]	0,604	0,661

V. RÉSULTATS ET DISCUSSION

A. Comparaison globale des modèles

On observe que les modèles de *gradient boosting*, en particulier CatBoost avec un QWK local de 0,662, fixent une ligne de base très solide. Le gain de performance s'explique par la robustesse de ces modèles face aux données tabulaires et par l'efficacité de la *variable split calibration*. L'évaluation sur l'ensemble de test indépendant de Kaggle a confirmé ces performances, avec un score public de 0,672 et un score privé de 0,675, attestant la capacité de généralisation du modèle et validant la méthode d'optimisation des seuils.

TabM[†] (avec *embeddings* PiecewiseLinear et optimisation bayésienne sur 30 *runs* W&B) atteint un QWK de **0,661** sur l'ensemble de validation, soit un score quasi équivalent à celui de CatBoost. À titre de comparaison, le MLP de référence plafonne à 0,632. L'apprentissage profond spécialisé pour les données tabulaires rivalise donc avec les meilleures méthodes de *gradient boosting* sur ce problème de régression ordinale.

On peut également constater que TabNet est au bas de la liste avec un QWK local de 0,529.

Cependant, quelle que soit l'approche considérée, qu'il s'agisse de l'ajout de données synthétiques ou de l'architecture *teacher-student*, il reste difficile d'obtenir des performances significativement supérieures à celles des modèles de *gradient boosting*. La stratégie B, utilisant GPT comme générateur de données synthétiques, semble dépasser très légèrement les résultats des modèles de boosting (QWK de 0,670). Cependant, l'écart reste trop faible pour pouvoir en tirer des conclusions solides. Au vu des résultats obtenus avec les autres méthodes de génération de données synthétiques, cette amélioration peut davantage être interprétée comme un résultat ponctuel favorable que comme une amélioration nette, robuste et réellement significative.

Finalement, les méthodes récentes testées demeurent néanmoins intéressantes, car elles parviennent à atteindre des ordres de grandeur similaires aux meilleures approches classiques sur données tabulaires. Mais, au regard de leur complexité, du temps d'entraînement nécessaire, de la difficulté de réglage des architectures et du coût expérimental associé, leur utilisation n'apparaît pas nécessairement rentable dans ce cadre. Les gains observés restent faibles, voire inexistant, et ne permettent pas

de justifier clairement l'abandon des méthodes de *gradient boosting*, qui offrent un compromis plus favorable entre performance, simplicité et robustesse.

B. Comparaison des méthodes de génération de données synthétiques

Plusieurs métriques ont été utilisées afin d'évaluer la qualité des données synthétiques, à la fois du point de vue de leur utilité pour un modèle en aval et de leur similarité avec les données réelles.

a) *QWK (Quadratic Weighted Kappa)*: Le QWK mesure l'accord entre les prédictions du modèle et les vraies classes, en tenant compte de la nature ordinale de la variable cible. Il est défini comme :

$$\kappa = 1 - \frac{\sum_{i,j} w_{i,j} O_{i,j}}{\sum_{i,j} w_{i,j} E_{i,j}}$$

où O est la matrice de confusion observée, E la matrice attendue et $w_{i,j}$ un poids quadratique pénalisant davantage les erreurs éloignées. Nous reportons :

- le QWK sur données réelles seules (référence) ;
- le QWK après augmentation ;
- Δ QWK, le gain apporté par les données synthétiques.

b) *AUC réel vs synthétique*: Cette métrique mesure la capacité d'un classifieur à distinguer les données réelles des données synthétiques. Un modèle est entraîné pour prédire si une ligne est réelle ou synthétique, puis l'AUC est calculée :

- AUC proche de 0,5 : les données synthétiques sont difficiles à distinguer (bon signe) ;
- AUC élevée : les données synthétiques sont facilement détectables.

c) *Quality Report (SDMetrics)*: Un rapport de qualité est calculé à l'aide de SDMetrics, fondé sur trois composantes :

- **Quality overall** : score global de similarité ;
- **Column shapes** : similarité des distributions marginales ;
- **Column pair trends** : préservation des corrélations entre variables.

Les résultats agrégés sont présentés dans le Tableau IV.

TABLE IV
COMPARAISON DES MÉTHODES DE GÉNÉRATION DE DONNÉES SYNTHÉTIQUES (STRATÉGIE B).

Modèle	Δ QWK	Quality
GPT (LLM)	+0,1419	0,9586
TabSyn	+0,0093	0,9231
CTGAN	−0,0235	0,8757
TVAE	−0,0204	0,8884

Dans un premier temps, nous avons évalué les différentes méthodes de génération selon plusieurs stratégies avec les métriques précédentes, avant d'estimer leur impact sur l'entraînement des modèles. Le Tableau IV présente une première comparaison pour la stratégie B.

Ces résultats constituent des observations préliminaires. On observe des écarts de performance entre méthodes, tant en termes de Δ QWK que de qualité des données générées. Plus

précisément, le modèle GPT obtient un gain de performance nettement plus élevé que les autres approches ; TabSyn affiche une amélioration plus modérée, tandis que CTGAN et TVAE conduisent à une dégradation des performances dans ce cadre expérimental.

Il est intéressant de noter que le modèle de langage utilisé (GPT-5.3) obtient de meilleures performances que des modèles spécifiquement conçus pour la génération de données tabulaires, malgré une approche reposant uniquement sur le *prompting*.

Dans un second temps, plusieurs stratégies de génération ont été explorées avec différents modèles. On a mesuré l’impact des données synthétiques sur l’entraînement et les performances en conditions réelles. Ces analyses sont détaillées dans les sections suivantes.

C. Comparaison des performances des modèles de deep learning sur données réelles et données synthétiques

On a commencé par évaluer nos différents modèles pour la stratégie A et la stratégie B. Aucun des modèles ni aucune des stratégies ne semblait prendre le devant : on n’a pas réellement observé de différence significative. De plus, aucun des modèles ni aucune des premières stratégies explorées n’avait une réelle utilité et ne parvenait à dépasser TabM sans données synthétiques.

On a ensuite utilisé GPT-5.3, qui a semblé se différencier des autres méthodes, sans pour autant améliorer significativement les résultats. L’approche teacher-student, bien que méthodologiquement réfléchie, ne semble pas non plus significativement meilleure que les autres. GPT-5.3 reste quand même la méthode la plus intéressante, ce qui confirme nos résultats précédents, même si elle ne l’est pas significativement.

Comme la méthode de génération avec des LLM semblait se démarquer, on a testé les deux autres stratégies, C et D, seulement avec cette méthode, puisque les autres semblaient vraiment peiner à fonctionner correctement.

Ces deux dernières stratégies ne semblent pas avoir donné de meilleurs résultats. Globalement, la meilleure stratégie dégagée semble être la B, ce qui indique que le facteur le plus important reste probablement le déséquilibre entre les classes.

De plus, on peut dire qu’il est surprenant qu’une méthode de génération de données synthétiques reposant sur un LLM en *zero-shot* obtienne de meilleurs résultats que des modèles spécialisés en génération tabulaire. Cela peut s’expliquer par le pré-entraînement massif des LLM, qui leur permet d’exploiter des régularités générales, même sur des tâches peu adaptées à leur objectif initial. À l’inverse, cette tâche reste difficile et les modèles conçus pour la génération tabulaire présentent encore des résultats mitigés dans notre cadre expérimental.

D. Étude d’ablation TabM

1) *Protocole*: Six variantes de TabM sont entraînées sur *Prudential* ($N = 59\,381$, $d = 130$ variables numériques après nettoyage) avec une division stratifiée 80 % / 20 % entraînement / validation. L’optimiseur est AdamW ($\eta = 10^{-3}$, $\lambda = 10^{-3}$), avec un *scheduler* CosineAnnealingLR et un *early stopping* de patience 10. La taille de lot est de 1024 ; les variantes $\dagger$

TABLE V
COMPARAISON DES PERFORMANCES DES MODÈLES DE DEEP LEARNING SUR DONNÉES RÉELLES ET DONNÉES SYNTHÉTIQUES

Modèle	QWK _{brut}	QWK _{+offset}
TabM (sans synthétique)	0,604	0,661
GPT (Stratégie D)	0,639	0,644
GPT (Stratégie C)	0,642	0,652
GPT (Stratégie B)	0,667	0,670
TabSyn (Stratégie B)	0,649	0,653
TabSyn (Stratégie A)	0,635	0,643
CTGAN (Stratégie B)	0,646	0,654
CTGAN (Stratégie A)	0,636	0,645
TVAE (Stratégie B)	0,642	0,649
TVAE (Stratégie A)	0,636	0,644
Teacher-student	0,656	0,660

utilisent $B = 48$ intervalles et $d_e = 16$ pour les *embeddings* PiecewiseLinear. Les prédictions continues sont post-traitées par une optimisation d’*offsets* par classe qui maximise le QWK sans ré-entraînement.

TABLE VI
RÉSULTATS DE L’ABLATION TABM SUR PRUDENTIAL (VALIDATION). PL : EMBEDDINGS PIECEWISELINEAR.

Variante	arch	Embed.	QWK _{brut}	QWK _{+off.}
MLP plain (référence)	plain	–	0,594	0,632
TabM-packed	tabm-packed	–	0,593	0,637
TabM-mini	tabm-mini	–	0,593	0,645
TabM	tabm	–	0,607	0,645
TabM-mini $\dagger$	tabm-mini	PL	0,608	0,656
TabM$\dagger$	tabm	PL	0,604	0,661

2) *Résultats*: L’ordre de performance est conforme aux prédictions de l’article : TabM-packed < TabM-mini < TabM < TabM-mini $\dagger$ < TabM $\dagger$. Le gain apporté par les *embeddings* PiecewiseLinear est le plus marqué (de +0,011 à +0,016 QWK selon la variante), ce qui confirme que la modélisation explicite des non-linéarités numériques est le facteur clé sur ce jeu de données. La régularisation par *BatchEnsemble* apporte un gain de l’ordre de +0,005 à +0,013 QWK selon la comparaison considérée. L’optimisation des *offsets* apporte un gain systématique d’environ +0,04 à +0,06 QWK sans aucun ré-entraînement.

3) *Sweep bayésien*: Un *sweep* bayésien (Weights & Biases, 30 *runs*) est conduit sur la variante TabM $\dagger$ pour optimiser les hyperparamètres η , λ , le *dropout*, B , d_e et n_{blocs} . Le meilleur *run* (*wise-sweep-22*) atteint un QWK de **0,6608** avec la configuration : $\eta = 1,12 \times 10^{-3}$, $\lambda = 3,37 \times 10^{-4}$, *dropout* = 0,229, $B = 48$, $d_e = 32$, $n_{\text{blocs}} = 3$. La substitution de $d_e = 16$ (valeur par défaut) par $d_e = 32$ est le seul changement substantiel, expliquant l’essentiel du gain de +0,001 par rapport à l’ablation.

4) *Ensemble stacking*: Afin d’évaluer si la combinaison de modèles hétérogènes peut dépasser TabM $\dagger$ seul, nous entraînons un ensemble par *stacking* à deux niveaux en validation croisée stratifiée ($k = 5$) :

— **Niveau 0** : TabM $\dagger$ (hyperparamètres optimaux du *sweep*), XGBoost ($n_{\text{est}} = 800$, $\eta = 0,05$, *depth* = 6), LightGBM

TABLE VII
TOP-5 RUNS DU SWEEP BAYÉSIEEN (TABM[†]).

Run	QWK _{off.}	η	d_e	n_b
wise-sweep-22	0,6608	$1,12 \times 10^{-3}$	32	3
mild-sweep-18	0,6605	$5,67 \times 10^{-4}$	32	3
blooming-sweep-10	0,6591	$8,23 \times 10^{-4}$	8	3
avid-sweep-7	0,6590	$1,46 \times 10^{-3}$	32	2
lyric-sweep-24	0,6590	$7,48 \times 10^{-4}$	8	3

($n_{\text{est}} = 800$, $\eta = 0,05$, 63 feuilles) ;

- **Niveau 1** : moyenne pondérée optimisée ($w_{\text{TabM}} = 0,4$, $w_{\text{XGB}} = 0,3$, $w_{\text{LGB}} = 0,3$) suivie d’une optimisation des *offsets*.

TABLE VIII
RÉSULTATS DU STACKING EN VALIDATION CROISÉE OOF ($k = 5$).

Modèle	QWK _{brut}	QWK _{off.}
TabM [†] (OOF)	0,594	0,624
XGBoost (OOF)	0,611	0,635
LightGBM (OOF)	0,607	0,637
Stacking	0,606	0,637
TabM [†] (ablation, <i>full train</i>)	0,604	0,661

Le *stacking* ne dépasse pas TabM[†] entraîné sur la totalité des données d’entraînement (0,637 contre 0,660). Deux facteurs l’expliquent : (1) TabM[†] perd $\approx 0,036$ QWK en mode OOF (entraîné sur 80 % des données seulement, avec *early stopping*) ; (2) XGBoost et LightGBM sont très fortement corrélés entre eux ($r = 0,986$), ce qui réduit la diversité effective de l’ensemble. La corrélation entre TabM[†] et les modèles de *boosting* est quasi nulle ($r \approx 0,01$), confirmant leur complémentarité théorique ; mais TabM est trop affaibli par le régime OOF pour en tirer parti.

VI. CE QUI N’A PAS FONCTIONNÉ

Les méthodes de deep learning semblent être tenues en échec par les modèles de boosting, bien moins complexes, mais beaucoup mieux adaptés à ce type de problème tabulaire.

Les différentes stratégies utilisant les données synthétiques n’ont pas permis d’améliorer significativement les résultats des modèles de deep learning, que les données soient générées avec ou sans étiquettes, ajoutées directement aux données réelles ou utilisées dans une approche teacher-student.

Cela montre que les données synthétiques générées ne reproduisent pas assez bien la distribution réelle des données tabulaires. Elles peuvent sembler plausibles, mais elles n’apportent pas nécessairement d’information utile pour améliorer l’apprentissage.

L’hypothèse selon laquelle un grand volume de données synthétiques pourrait améliorer la généralisation n’a donc pas été validée. Ces résultats montrent les limites des méthodes actuelles de génération pour les données tabulaires.

Concernant la partie TabM, le *stacking* n’a pas permis de dépasser TabM[†] seul : l’affaiblissement de TabM en régime OOF ($-0,036$ QWK) annule le bénéfice attendu de la diversité.

Les *runs* supplémentaires du *sweep* bayésien au-delà de 30 n’apportaient plus de gain mesurable, les réglages par défaut étant déjà quasi optimaux.

VII. POSSIBILITÉS D’AMÉLIORATION

Pour la partie TabM, plusieurs pistes seraient envisageables : (1) augmenter le nombre d’*epochs* par *fold* lors du *stacking* (actuellement limité par l’*early stopping* sur 80 % des données) ; (2) introduire un modèle *tabm-packed* d’architecture différente pour augmenter la diversité de l’ensemble ; (3) explorer des valeurs de $k > 32$ ou des architectures à plus de blocs pour les variantes sans *embeddings*.

Bien que TabM[†] ait démontré une capacité supérieure à capturer des relations non linéaires complexes grâce à ses *embeddings* PiecewiseLinear, il demeure un modèle entraîné de manière purement supervisée sur un jeu de données spécifique. À l’inverse, TabDPT (*Tabular Decoding Pre-trained Transformer*) [8] exploite un pré-entraînement massif sur des milliers de jeux de données diversifiés, lui permettant de développer un biais inductif généralisé sur la structure intrinsèque des données tabulaires.

L’intégration de TabDPT dans notre architecture de *stacking* pourrait s’avérer bénéfique selon deux axes :

- 1) **Transfert de connaissances** : l’utilisation des représentations apprises par TabDPT comme variables d’entrée additionnelles pour le mélange final apporterait une perspective « globale » que les modèles entraînés uniquement sur *Prudential* ne possèdent pas ;
- 2) **Robustesse au régime OOF** : un modèle de fondation pré-entraîné tend à être plus résilient dans ces conditions, ce qui pourrait stabiliser le score global du *stacking*.

L’évaluation de cette synergie entre modèles de fondation et architectures spécialisées constitue la prochaine étape logique pour franchir les limites de précision actuelles de ce problème de classification ordinaire.

VIII. CONCLUSION

Ce projet a confronté les méthodes de *gradient boosting* (XGBoost, CatBoost) à des architectures d’apprentissage profond récentes (TabNet, TabM) pour la classification ordinaire du risque en assurance-vie, sur le jeu de données *Prudential Life Insurance Assessment*. Cinq conclusions se dégagent.

(1) **Le *gradient boosting* demeure la référence sur les données tabulaires ordinales.** L’association d’une décomposition en sept classifieurs binaires et d’une calibration conditionnelle des seuils inspirée des solutions gagnantes de la compétition Kaggle, permet à CatBoost d’atteindre le meilleur score absolu de notre étude (QWK = 0,662 en validation, Tableau III).

(2) **TabM constitue une alternative compétitive en apprentissage profond.** Avec un QWK de 0,661, quasi équivalent à CatBoost, la variante TabM[†] optimisée par recherche bayésienne (Section V, Tableau VII) surpasse de +0,029 le MLP de référence (0,632) et largement TabNet (0,529). Sur *Prudential*, c’est l’une des rares architectures neuronales spécifiquement conçues pour les données tabulaires qui soutient une comparaison directe avec le *gradient boosting*.

(3) Les *embeddings* `PiecewiseLinear` et le *BatchEnsemble* sont les deux composantes qui rendent `TabM` compétitif face au *boosting*. L'étude d'ablation (Tableau VI) isole précisément l'apport de chaque mécanisme : les *embeddings* `PiecewiseLinear` contribuent à hauteur de +0,011 à +0,015 QWK selon la variante, tandis que le *BatchEnsemble* apporte +0,013 QWK. La variante *packed* (sans partage des poids principaux) n'apporte en revanche aucun bénéfice net, suggérant que c'est la combinaison *partage paramétrique* + *adaptateurs légers* qui crée la diversité exploitable, et non le simple fait d'entraîner k MLP en parallèle.

(4) L'ajout de données synthétiques ne bonifie que marginalement les performances des approches de *deep learning*. Parmi les méthodes testées (Tableau V), seul GPT-5.3 sous la stratégie B produit un gain visible sur `TabM` (QWK de 0,670 contre 0,661 sans données synthétiques). Cet écart reste statistiquement modeste et ne suffit pas à dépasser significativement la ligne de base du *boosting*. CTGAN, TVAE et `TabSyn` dégradent même les performances sous certaines stratégies, suggérant que les générateurs tabulaires actuels reproduisent imparfaitement les distributions complexes de Prudential.

(5) Le *stacking* `TabM` + `XGBoost` + `LightGBM` n'a pas dégagé la synergie attendue. Avec un QWK de 0,637 (Tableau VIII), l'ensemble reste en deçà de `TabM`[†] entraîné sur la totalité des données (0,661). Deux causes l'expliquent : `TabM`[†] perd $\approx 0,036$ QWK en régime *out-of-fold* (entraîné sur 80 % des données), et `XGBoost` et `LightGBM` sont fortement corrélés entre eux ($r = 0,986$), réduisant la diversité effective du méta-modèle malgré la complémentarité théorique entre `TabM` et le *boosting* ($r \approx 0,01$).

En définitive, sur des données tabulaires ordinales hétérogènes comme Prudential, le *gradient boosting* reste la référence en performance brute, mais `TabM`, lorsqu'il intègre des *embeddings* spécialisés et un mécanisme d'ensemble paramétriquement efficace, parvient à l'égaliser sans surcoût d'entraînement majeur. L'écart résiduel justifie d'explorer d'autres voies — modèles de fondation tabulaires (`TabDPT`, `TabPFN v2`), *stacking* hétérogène mieux pondéré, ou diffusion conditionnelle pour la génération synthétique — qui pourraient permettre à l'apprentissage profond de devancer les méthodes classiques sur ce type de problème (Section VII).

IX. UTILISATION DE L'IA GÉNÉRATIVE

Conformément aux directives du cours, nous déclarons ici l'ensemble des usages d'outils d'IA générative ayant contribué à ce projet. Tous les contenus produits par ces outils ont été lus, vérifiés et adaptés par les auteurs, qui demeurent seuls responsables des choix méthodologiques, des résultats rapportés et de la rédaction du présent rapport.

— **Génération de données synthétiques tabulaires** : les modèles GPT-5.3 (OpenAI) [19], Gemini 3.1 Pro (Google DeepMind) [20] et Claude Opus 4.7 Anthropic) [21] ont été utilisés comme générateurs de données synthétiques pour l'augmentation du jeu *Prudential* (Section III, sous-section *Génération via LLM*). Les *prompts* ont été conçus

et itérés par les auteurs ; un exemple complet est fourni en Annexe X-A. Les données générées ont été contre-vérifiées via les métriques de qualité présentées au Tableau IV.

- **Assistance au développement** : aide ponctuelle pour l'écriture des requêtes API vers `Weights & Biases` (`get_sweep_config.py`) et pour la prise en main de bases de code complexes récupérées depuis GitHub, en particulier le dépôt officiel de `TabSyn` [18] (compréhension du fonctionnement, adaptation à notre jeu de données, recherche d'hyperparamètres). Le code produit a été relu, testé et intégré manuellement.
- **Assistance à la rédaction** : relecture, reformulation et vérification typographique de certaines sections du rapport. Aucune section n'a été rédigée intégralement par un modèle ; les choix de structure, d'argumentation et d'analyse sont ceux des auteurs.
- **Aide à la compréhension de la littérature** : clarification ponctuelle de concepts techniques abordés dans les articles cités (par exemple les *embeddings* `PiecewiseLinear`), en complément de la lecture des publications originales.

Aucun outil d'IA générative n'a été utilisé pour produire ou modifier les résultats expérimentaux rapportés (scores QWK, courbes d'entraînement, tableaux d'ablation), qui proviennent intégralement de nos exécutions.

X. ANNEXE

A. Prompt

Exemple de prompt utilisé pour générer des données synthétiques non annotées à l'aide d'un LLM.

```
You are a tabular data generation system. Your task is to
generate synthetic data for an insurance dataset
WITHOUT labels.

### DATASET ###
- Each row represents an individual.
- Features include numerical, categorical, and binary
  variables.
- Target variable in the original dataset: "Response" (
  integer from 1 to 8).
- An example dataset will be provided as input: you MUST
  learn its structure, column names, data types, value
  ranges, missing-value patterns, and feature
  relationships from it.
- Previously generated synthetic datasets will also be
  provided: you MUST NOT copy, reuse, or slightly modify
  these rows.

### OBJECTIVE ###
Generate new synthetic data WITHOUT the target variable.
The generated samples should mainly explore regions that
are LESS likely to be classified as Response = 8.
Focus on realistic profiles that could plausibly correspond
to Response classes 1 to 7, especially
underrepresented, ambiguous, or intermediate-risk
regions.

### REQUIREMENTS ###
Generate exactly 50000 synthetic rows:
- Do NOT include the "Response" column.
- Generate only input features.
- All rows must be NEW and independent.
- Do NOT overlap with existing real or synthetic data.
- Do NOT generate obvious high-confidence Response = 8
  profiles.
```

```

### TARGETED DISTRIBUTION ###
Prioritize:
- profiles likely to belong to classes 1 to 7;
- ambiguous profiles between neighboring classes;
- intermediate-risk samples;
- rare but realistic combinations;
- edge cases that are not extreme class-8-like profiles;
- underrepresented regions of the feature space.

Avoid:
- very typical Response = 8-like profiles;
- overly dominant high-risk profiles;
- repetitive patterns;
- trivial or easy samples;
- copying or slightly modifying existing rows.

### STATISTICAL CONSTRAINTS ###
Preserve global consistency with the original dataset:
- feature distributions;
- valid value ranges;
- data types;
- missing-value patterns;
- realistic feature correlations.

You may shift the generated distribution away from obvious
Response = 8 regions, but all samples must remain
realistic and coherent with the original dataset.

### HARD CONSTRAINTS ###
- Do not copy, extend, or approximate existing rows.
- Ensure all samples are novel.
- No impossible feature combinations.
- Keep values within valid ranges.
- Preserve exact column names.
- Preserve exact data types.
- Do not add extra columns.
- Do not include the "Response" column.

### OUTPUT FORMAT ###
CSV only.
No explanations.
No markdown.
No text before or after the CSV.

```

- [12] J. Yoon, Y. Zhang, J. Jordon, and M. van der Schaar, "Vime : Extending the success of self- and semi-supervised learning to tabular data," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [13] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv :1503.02531*, 2015.
- [14] P. Financial, "Prudential life insurance assessment," Kaggle Competition, 2015. [Online]. Available : <https://www.kaggle.com/c/prudential-life-insurance-assessment>
- [15] S. O. Arik and T. Pfister, "Tabnet : Attentive interpretable tabular learning," 2020. [Online]. Available : <https://arxiv.org/abs/1908.07442>
- [16] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [17] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional gan," in *Advances in Neural Information Processing Systems*, 2019.
- [18] H. Zhang, J. Zhang, B. Srinivasan, Z. Shen, X. Qin, C. Faloutsos, H. Rangwala, and G. Karypis, "Mixed-type tabular data synthesis with score-based diffusion in latent space," in *International Conference on Learning Representations (ICLR)*, 2024.
- [19] OpenAI, "GPT-5.3 Instant : Smoother, more useful everyday conversations," <https://openai.com/index/gpt-5-3-instant/>, 2026, accessed : 2026-05-02.
- [20] Google DeepMind, "Gemini 3.1 Pro," <https://deepmind.google/models/gemini/pro/>, 2026, accessed : 2026-05-02.
- [21] Anthropic, "Claude Opus 4.7," <https://www.anthropic.com/claude/opus>, 2026, accessed : 2026-05-02.

RÉFÉRENCES

- [1] T. Chen and C. Guestrin, "Xgboost : A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [2] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm : A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [3] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "Catboost : Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [4] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why tree-based models still outperform deep learning on tabular data," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [5] S. Ö. Arik and T. Pfister, "Tabnet : Attentive interpretable tabular learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.
- [6] Y. Gorishniy, I. Rubachev, A. Shlenskii, A. Dyakonov, and A. Babenko, "Tabm : Advancing tabular deep learning with parameter-efficient ensembling," *arXiv preprint arXiv :2410.24210*, 2024.
- [7] N. Hollmann, S. Müller, K. Eggenberger, and F. Hutter, "Tabpfn : A transformer that solves small tabular classification problems in a second," in *International Conference on Learning Representations (ICLR)*, 2023.
- [8] Y.-X. Wang and et al., "Tabdpt : Scaling tabular foundation models with deep pseudo-target training," *arXiv preprint arXiv :2410.18413*, 2024.
- [9] E. Frank and M. Hall, "A simple approach to ordinal classification," *European Conference on Machine Learning (ECML)*, 2001.
- [10] L. Hou, C.-P. Yu, and D. Samaras, "Squared earth mover's distance-based loss for training deep neural networks," in *arXiv preprint arXiv :1611.05916*, 2016.
- [11] D.-H. Lee, "Pseudo-label : The simple and efficient semi-supervised learning method for deep neural networks," in *ICML Workshop on Challenges in Representation Learning*, 2013.